

Presentation Guideline — Intelligent Recruitment System

For: Academic Project Defense Panel | Slot: 10-15 minutes

This guideline is written to be used directly during your defense. It gives you exact talking points, suggested timing, a live-demo click path through the real running system, and answers to the questions a panel is most likely to ask. Practice it out loud at least twice before the real thing — 10-15 minutes disappears fast once you start explaining things.

1. Time Budget (keep a clock visible while practicing)

| Section | Time | Cumulative | |---|---|---| | Opening & problem statement | 1 min | 1 min | | Objectives & scope | 1 min | 2 min | | System architecture (high level) | 1.5 min | 3.5 min | | Live demo (three roles) | 6 min | 9.5 min | | AI innovation highlight | 2 min | 11.5 min | | Conclusion & challenges faced | 1.5 min | 13 min | | Buffer for Q&A transition | 2 min | 15 min |

If you're cut short, protect the **live demo** and the **AI highlight** — those are what differentiate this project from a generic CRUD app and are what panels remember.

2. Opening (say this, don't read it — memorize the shape of it)

"Good [morning/afternoon]. I built an Intelligent Recruitment System — a web platform that uses AI to screen and rank job applicants automatically, instead of a recruiter reading every CV by hand. It serves three types of users: applicants who apply for jobs, recruiters who manage vacancies and hiring, and an administrator who oversees the whole platform."

Problem statement (30 sec): Manual CV screening is slow, inconsistent, and hard to scale — a recruiter reviewing 100 applications for one role spends hours doing pattern-matching a computer can do in seconds. This project automates that first-pass screening while keeping the final hiring decision with a human.

3. Objectives (state 3-4, not more — a panel will ask you to justify each one)

1. Let applicants browse jobs, apply, and track their application status end-to-end online.
 2. Let recruiters post vacancies and get an AI-generated match score, ranking, and reasoning for every candidate, instead of scoring manually.
 3. Give recruiters a re-scan option so they aren't locked into one AI verdict if they disagree with it — AI is decision **support**, not a decision **maker**.
 4. Give an administrator full oversight: user management, system-wide analytics, audit logs, and governance controls (e.g. auto-closing expired vacancies).
-

4. System Architecture (keep this to a diagram + one sentence per layer, don't read code)

Stack: Laravel 13 (PHP 8.3) on the backend, MySQL for data, Blade + Tailwind for the interface, Google's Gemini API for the AI layer.

Say something like:

"The system has three layers: a Laravel backend that handles business logic and security, a MySQL database that stores applicants, jobs, applications, interviews, and scores, and an AI layer that calls Google's Gemini language model to read resumes and job descriptions and produce a match score with reasoning."

If asked to draw it: **Browser → Laravel Controllers → [Scoring Service ⇌ Gemini API] → MySQL**, with a background queue handling the AI calls so the interface doesn't freeze while waiting on a response.

Mention briefly: the system supports **English and Swahili**, switchable at any time, across all three portals — a small but concrete detail that shows attention to real-world usability for a local audience.

5. Live Demo — suggested click path (this is the core of your 15 minutes)

Rehearse this exact sequence so you're never hunting for a button live. Have test accounts for all three roles logged in on separate browser tabs beforehand — don't log in/out during the defense, it wastes time.

A. Applicant view (2 min)

1. Show the job listing page → open one vacancy → point out the requirements shown to the applicant.
2. Show "My Applications" with a real AI match score already sitting on a submitted application.
3. Open the AI chatbot, type a natural question (e.g. *"which jobs are open right now?"* or *"am I a good fit for this role?"*) — show that it answers using the applicant's **real data**, not a canned script.

B. Recruiter/HR view (2.5 min)

1. Open the candidate ranking page for a job → show candidates sorted by AI score.
2. Open one candidate's **AI Results** page → walk through: match %, matched vs missing skills, strengths/weaknesses, GPA analysis, and the written AI explanation.
3. Point at the **"Not satisfied? Re-scan with AI"** button — explain this is a deliberate design choice: *"the AI's first answer isn't final — the recruiter can trigger a fresh evaluation if they disagree."*
4. Schedule (or show an already-scheduled) interview → mention the applicant gets a real email invitation.

C. Admin view (1.5 min)

1. Show the admin dashboard: total users, active jobs, applications, recruiter count, system health tiles.
2. Show the analytics charts (applications by department, recruitment trends).
3. Briefly show recruiter account management (create/manage/reset password/remove).

Tip: narrate what you're clicking *before* you click it ("now I'll open the candidate ranking...") — a silent click-through loses a panel's attention fast.

6. AI Innovation Highlight (2 min — this is what makes the project defensible as "AI", not just a form app)

Don't just say "we used AI." Be specific — panels probe generic AI claims hardest:

"Every scoring decision is generated by a structured prompt sent to Gemini, forced to return a fixed JSON schema — overall score, matched skills, missing skills, strengths, weaknesses, risk factors, and a written explanation. The AI is instructed to reason step by step like an experienced recruiter would: check required skills coverage, weigh relevant experience, check education and GPA fit, and only then reach an overall verdict — so a candidate missing a critical required skill can't score high just because other areas look good."

Mention the **hybrid scoring model**: a deterministic score computed from structured data (skills match, years of experience, GPA) is blended with the AI's independent judgment (55/45 weighting) — so the AI isn't the *only* signal, it's calibrated against a transparent, explainable baseline. This answers the inevitable "how do you know the AI isn't just making things up?" question before it's asked.

Also worth one sentence: the chatbot is **grounded** — it's given the applicant's or recruiter's actual database records as context and instructed never to invent facts not present in that data, so it answers real questions about real applications instead of giving generic scripted replies.

7. Challenges Faced & How You Solved Them (1.5 min — panels respect this section, it shows real engineering)

Pick 2–3 honest, concrete ones. Suggested (these are real issues solved during development):

- **AI rate limits.** The free-tier AI API caps requests per day/minute. Solved with automatic retry logic that respects the API's own suggested wait time, plus a deterministic fallback score so the system degrades gracefully instead of breaking when the AI is temporarily unavailable.
- **Long AI response times vs server timeouts.** Some AI responses (with reasoning enabled) can take 30–45 seconds. Solved by extending the execution time limit specifically for AI calls so legitimate slow responses aren't killed mid-request.
- **Bilingual coverage.** Early on, language switching only translated the navigation sidebar, not page content. Solved by systematically auditing every page and wiring all user-facing text through Laravel's translation system, in both English and Swahili.

8. Anticipated Panel Questions — with answers

Q: What happens if the AI service is down or gives a wrong answer? A: The system falls back to a deterministic score computed from structured data (skills match, years of experience, education, GPA) — so scoring never fully fails. And recruiters can always re-scan or override the AI's recommendation; the AI never makes the final hiring decision.

Q: How do you prevent the AI from being biased or unfair? A: The scoring prompt explicitly instructs the model to ground every judgment in specific evidence from the candidate's actual data — skills, projects, certifications — and forbids inventing or assuming attributes not present in the data. It's also decision-support only: every AI recommendation is reviewable and reversible by a human recruiter.

Q: Why Gemini specifically, and not a custom-trained model? A: Training a custom model needs a large labeled hiring dataset we don't have. A general-purpose LLM with a carefully engineered prompt gives strong reasoning quality immediately, without needing to collect and label thousands of past hiring decisions first — appropriate for the scope and timeline of this project.

Q: How is candidate data kept secure / how do you handle privacy? A: Access is enforced at two levels. First, route middleware checks the logged-in user's role on every request and blocks anyone who isn't an applicant, recruiter, or admin from reaching a portal that isn't theirs. Second, within the recruiter portal specifically, a recruiter can only open an application

if they own the job it was submitted to — every application-detail action checks the job's assigned recruiter ID against the logged-in user and returns a 403 if they don't match. Only Admin/HR-manager roles can see across all recruiters' applications. Passwords are hashed, and recruiter actions (status changes, account creation, etc.) are written to an audit log with actor, action, and timestamp.

Q: Is this actually deployed / production-ready, or just a prototype? A: Be honest here. If it's still a local/demo deployment, say so plainly: *"This is currently running as a functional demo — the core screening, scoring, scheduling, and notification flows all work end-to-end with real data, but it hasn't been deployed to a production server yet."* Panels respect honesty far more than an over-claimed "it's production-ready" that falls apart under one follow-up question.

Q: What would you improve if you had more time? A: Have 2–3 ready, e.g.: a dedicated transactional email service instead of a personal Gmail SMTP account for higher send volume; a version-history view so recruiters can compare an AI re-scan against the previous result instead of only seeing the latest one; expanding the skill-matching vocabulary further for specialized/niche roles.

9. Closing line (memorize this, end on it, don't trail off)

"In summary, this system takes a manual, hours-long screening process and turns it into an AI-assisted workflow that takes seconds per candidate — while keeping a human recruiter in control of every final decision. Thank you, I'm happy to take questions."

10. Pre-defense checklist

- [] Test accounts ready and logged in for all three roles (applicant, recruiter/HR, admin) — don't log in live.
- [] At least one application already has a real AI score visible (don't rely on live scoring during the defense — API calls can be slow or rate-limited at the worst moment).
- [] Confirm the AI service is actually reachable right now (quota not exhausted) — test it 30 minutes before, not the morning before.
- [] Language switch (EN/SW) tested and working on the exact pages you'll demo.
- [] Know your own answer to the "is this production-ready" and "how is data kept secure" questions cold — these are asked in nearly every defense.
- [] Time yourself once, fully, out loud, with a clock. Cut ruthlessly if you're over 13 minutes — leave real room for questions.